

Florence Hall

PostgreSQL 로 만든 좌석 예매 사이트

데이터베이스 과제

202101248

유소정

무엇을 만들었나요

공연 좌석 예매 사이트를 만들었습니다.

- 회원가입 / 로그인
- 공연 목록 → 상세 → 좌석 선택 → 예매
- 마이페이지에서 예매 내역 확인 / 취소
- 통계 페이지 (점유율, 결제 TOP 5, 일별 추이)

같은 좌석을 거의 같은 시간에 두 사람이 누르면 어떻게 될까요?
이 시나리오를 시연해보고 싶어서 도메인을 골랐습니다.

웹 서비스와 DBMS 가 어떻게 연동되나요

```
브라우저
  | HTTP / FormData
  ▼
Next.js Server Action ('use server')
  | 함수 호출
  ▼
src/domain/*.ts (SQL + 비즈니스 규칙)
  | pg.Pool / withTransaction
  ▼
PostgreSQL 16
```

- **Server Action** 이 요청 진입점 — 클라이언트가 함수처럼 호출합니다
- **도메인 계층**이 SQL 과 트랜잭션 경계를 정의합니다
- **pg 드라이버**가 PostgreSQL 과 직접 통신합니다

별도 백엔드 프로세스(Express 등) 없이 한 프로젝트 안에서 풀스택을 처리합니다.

왜 좌석 예매 도메인인가요

과제 평가 항목이 릴레이션 · 쿼리 · 트랜잭션 세 가지였습니다.

이 셋이 모두 자연스럽게 등장하는 가장 단순한 도메인이 좌석 예매라고 생각했습니다.

평가 항목	좌석 예매가 보여주는 것
Relation	users ↔ bookings ↔ performances 관계
Query	점유율, 결제 TOP 5, 일별 추이 등 다양한 집계
Transaction	동시 예매 충돌을 막는 FOR UPDATE + partial UNIQUE

특히 트랜잭션은 없으면 어떻게 깨지는지 시연이 직관적입니다.

기술 스택

- [Next.js 16](#) (App Router) + React 19 + TypeScript
- [PostgreSQL 16](#)
- DB 는 [pg](#) 드라이버로 직접 — [ORM 은 쓰지 않았습니다](#)
- 인증은 bcryptjs + HMAC 쿠키 세션 (직접 구현)
- 테스트: Vitest(통합) + Playwright(E2E)
- 성능 측정: [pgbench](#) (강의에서 배운 도구)

별도 백엔드 프로세스는 없습니다. Server Actions 로 풀스택을 한 프로젝트 안에서 처리했습니다.

ORM 을 쓰지 않은 이유

평가 항목이 *"쿼리·트랜잭션을 어떻게 정의했는지"* 인데,
ORM 이 SQL 을 가리면 어필이 어렵습니다.

생짜 SQL 로 작성하면 다음이 코드에서 그대로 보입니다.

- `SELECT ... FOR UPDATE` 의 정확한 위치와 의도
- partial UNIQUE 인덱스 활용
- 트랜잭션 경계의 명확한 표시

학습 단계에서는 ORM 없이 직접 쓰는 게 훨씬 낫다고 생각합니다.

데이터 모델

테이블은 3개입니다: `users`, `performances`, `bookings`.

```
users      (id, email UNIQUE, name, password_hash, ...)  
  
performances (id, title, artist, performed_at, price, ...)  
  
bookings   (id, booking_group_id, user_id, performance_id,  
            section, row_label, seat_number,  
            status, price_paid, ...)
```

좌석을 별도 테이블로 두지 않고 `bookings` 행에 좌표(`section`, `row_label`, `seat_number`)로 직접 박았습니다.

처음엔 테이블이 5개였습니다

처음 설계: `users`, `performances`, `seats`, `bookings`, `booking_seats`

junction table 까지 두니까 발표 때 "각 테이블이 왜 있는지" 설명이 길어졌습니다.

3개로 줄이면서 깨달은 점:

- 모든 공연이 같은 좌석 배치 → 좌석 마스터는 코드 상수로 충분
- junction table 은 좌석 좌표를 행에 박으면 사라집니다
- 한 번에 여러 좌석을 예약하면 `booking_group_id` 로 묶습니다

정규화 과잉도 안티패턴이 될 수 있습니다.

핵심: partial UNIQUE 인덱스

```
CREATE UNIQUE INDEX uniq_confirmed_seat
  ON bookings (performance_id, section, row_label, seat_number)
  WHERE status = 'CONFIRMED';
```

이 한 줄의 정의로 다음을 한꺼번에 처리합니다.

- **활성 예매에만 적용** → 같은 좌석에 활성 예매가 둘일 수 없습니다
- **취소된 행은 자동으로 빠짐** → 같은 좌석 재예매가 가능합니다

"한 좌석 = 한 사람, 취소 시 풀림" 규칙이
DB 레벨에서 강제됩니다.

예매 트랜잭션

`src/domain/bookings.ts` 에 있습니다. 한 트랜잭션 안에서 잠금·검사·발급·INSERT 를 처리합니다.

```
BEGIN;

SELECT price FROM performances WHERE id = $1;

SELECT section, row_label, seat_number
  FROM bookings
 WHERE performance_id = $1 AND status = 'CONFIRMED'
 FOR UPDATE;
-- 선택한 좌석 중 이미 잡힌 게 있으면 ROLLBACK

SELECT nextval('booking_group_id_seq');

INSERT INTO bookings (...) SELECT ...;

COMMIT;
```

동시성이 어떻게 보장되나요

사용자 A 트랜잭션

BEGIN;

SELECT FOR UPDATE; ← 잠금

INSERT, COMMIT;

사용자 B 트랜잭션

BEGIN;

SELECT FOR UPDATE;

← A 가 끝날 때까지 대기

← 대기 해제, A 가 잡은 좌석 발견

BookingConflictError

ROLLBACK;

FOR UPDATE 가 두 번째 트랜잭션을 직렬화하고,
partial UNIQUE 가 마지막 안전망 역할을 합니다.

말로만 보장하지 않습니다 — 테스트로 검증

```
it("두 사용자가 동시에 같은 좌석을 잡으면 한쪽만 성공한다", async () => {
  const [a, b] = await Promise.allSettled([
    bookSeats({ userId: userA, performanceId, seats: [A1_1] }),
    bookSeats({ userId: userB, performanceId, seats: [A1_1] }),
  ]);
  expect([a, b].filter(r => r.status === "fulfilled")).toHaveLength(1);
  expect([a, b].filter(r => r.status === "rejected")).toHaveLength(1);
});
```

`Promise.allSettled` 로 두 트랜잭션을 정말 동시에 띄우고,
정확히 하나만 성공하는지 매 빌드마다 확인합니다.

자동화 테스트 결과

Vitest 통합 테스트 — 실제 DB 연결, 10개 케이스

- 예매 / 충돌 / 동시성 / 취소 / 권한
- 통계 쿼리 정확성

Playwright E2E — 실제 Chromium 브라우저, 7개 시나리오

- 회원가입 → 좌석 선택 → 예매 → 마이페이지 흐름
- 취소 후 좌석 재예매 가능 확인
- 비로그인 시 예매 버튼 비활성화

전체 17개 테스트가 모두 통과합니다.

성능 측정 — pgbench

강의에서 배운 PostgreSQL 표준 도구로 측정했습니다.

`bench/book_workload.sql` 에 예매 트랜잭션을 작성하고
시나리오별로 `-c`, `-T` 옵션을 바꿔가며 실행했습니다.

시나리오	클라이언트	TPS	평균 Latency
Baseline	1	9,001	0.11 ms
동시성 (정상)	10	10,691	0.88 ms
고부하	50	8,595	4.96 ms

측정 결과 해석

-c 1 → 10 : TPS 가 19% 증가 (병렬 처리 효과)

-c 10 → 50 : TPS 가 오히려 감소

- 잠금 경합이 빈번해집니다
- 컨텍스트 스위칭 비용이 누적됩니다

Latency 는 동시성에 거의 선형으로 비례합니다.

클라이언트	1	10	50
Latency	0.11 ms	0.88 ms	4.96 ms
증가 배수	1×	8×	45×

동시성을 무한정 늘린다고 처리량이 늘지는 않습니다.

흥미로운 발견: 인덱스 ON vs OFF

`idx_bookings_performance` 인덱스를 DROP 한 상태로 같은 워크로드를 측정해보았습니다.

상태	TPS	평균 Latency
인덱스 ON	11,048	0.905 ms
인덱스 OFF	11,297	0.885 ms

INSERT 위주 워크로드에서는 인덱스 ON 이 *오히려* 약간 느렸습니다.

인덱스 유지 비용이 INSERT 마다 들어가기 때문입니다.

실서비스는 SELECT 비중이 크니 결과가 반대로 나오겠지만,
"인덱스 = 무조건 빠름" 이 아니라는 점을 실측으로 확인했습니다.

통계 쿼리들

`/stats` 페이지에서 시각화됩니다. SQL 표현력을 보여주기 좋은 쿼리들입니다.

공연별 점유율 — `LEFT JOIN` + `GROUP BY` + `FILTER`

```
COUNT(b.id) FILTER (WHERE b.status = 'CONFIRMED')
```

결제 TOP 5 — `JOIN` + `SUM` + `COUNT(DISTINCT)`

```
SUM(price_paid), COUNT(DISTINCT booking_group_id)
```

일별 추이 — `date_trunc` + `GROUP BY`

```
date_trunc('day', created_at)
```

폴더 구조 — 책임 분리

```
src/  
  lib/      인프라 (DB 풀, 트랜잭션 헬퍼, 세션, 비밀번호)  
  domain/   SQL 과 비즈니스 규칙 (모든 SQL 은 여기)  
  app/      Next.js 라우트 + 서버 액션  
db/         schema.sql, seed.sql  
e2e/        Playwright 시나리오  
bench/      pgbench 워크로드  
docs/       결과보고서, 발표 슬라이드
```

원칙은 단순합니다.

lib 는 라이브러리 래퍼만 / **domain** 은 SQL 과 비즈니스 / **app** 은 렌더링과 액션.

한 컴포넌트에 여러 책임을 몰지 않으려고 했습니다.

회고

1. 정규화도 과하면 안티패턴이 될 수 있습니다.

5 테이블 → 3 테이블로 줄이며 발표 분량과 이해도가 함께 정리되었습니다.

2. ORM 없이 SQL 직접 쓰는 게 학습엔 훨씬 유리합니다.

EXPLAIN 보기도 쉽고, 트랜잭션 경계가 코드에서 그대로 보입니다.

3. 측정해봐야 직관이 맞는지 알 수 있습니다.

"인덱스 = 빠름" 이라는 직관이 INSERT 워크로드에서는 틀렸음을 pgbench 로 확인했습니다.

감사합니다

202101248

유소정